DEVOPS ENGINEER
REAL-TIME SCENARIO INTERVIEW GUIDE

Prepared from the attached resume and Senior DevOps Engineer job description

Focus areas: Jenkins multi-environment CI/CD, Kubernetes/EKS operations and upgrades, Ansible (including Windows), AWS infrastructure, Terraform, Docker, monitoring, security, migration and production incident handling.

How to use this guide

Read the scenario, understand the decision sequence, and practise explaining the answer aloud. The answers are written in interview-friendly first person. Keep only statements that match work you have actually performed. Where the resume says “managed”, “designed”, “provisioned” or “supported”, be ready to explain the exact task, environment, command, evidence and your level of ownership.

Important alignment note

Your resume highlights on-premises Kubernetes for Jio Enterprise Portal and AWS/EKS/ECS for Jio Selfcare Portal. The JD additionally expects production EKS, strong Ansible (especially Windows), Azure-to-AWS migration and data-platform services such as Glue, SQS/SNS, EventBridge, Athena and Lake Formation. This guide prepares you for those topics, but it does not present unlisted tools as proven hands-on experience. For a gap, explain the transferable concept and say clearly what you have and have not operated.

Resume technologies used: AWS (EC2, VPC, IAM, S3, EKS, ECS, ECR, RDS, Lambda, CloudWatch), Kubernetes, Docker, Jenkins, GitHub, Maven, SonarQube, Harness, Terraform modules/workspaces, Ansible, Bash, Python, YAML, Prometheus, Grafana, MySQL, Azure DevOps, Linux.

1. Jenkins CI/CD across Development, Staging and Production

Q. How would you design one Jenkins pipeline for Dev, Staging and Production?

Suggested answer: I would keep one version-controlled Jenkinsfile and make the target environment a controlled pipeline parameter or promotion stage. The pipeline would: (1) checkout a tagged/approved Git revision, (2) compile and run unit tests with Maven, (3) run SonarQube quality checks, (4) build one Docker image, (5) scan and tag it with an immutable build number or Git commit, (6) push it to the registry, and (7) deploy that same image digest to Dev, then Staging, then Production. Environment-specific values such as replicas, URLs and resource limits belong in separate configuration files or deployment values, not in different application images. Production should require an approval gate and a restricted Jenkins credential. After each deployment I would verify rollout status, pod readiness, service endpoints and application health. This matches the resume's Jenkins/GitHub/Maven/SonarQube/Docker and multi-environment Kubernetes work.

Q. How do you prevent Dev, Staging and Production from running different code?

Suggested answer: I build the image once and promote the exact same immutable image through environments. I record the Git commit, build number, image tag and preferably image digest in the build metadata. The deployment stage changes only environment configuration and approved secrets. I do not rebuild the image separately for Production, because that can introduce an untested artifact.

Q. How would you handle production approval and access?

Suggested answer: I would use a Jenkins input/approval step before Production, restrict who can approve it, and use a dedicated least-privilege service account or credential for deployment. Secrets should be stored in Jenkins Credentials or an approved secrets manager, never hard-coded in the Jenkinsfile or printed in logs. I would also ensure the production branch/tag and deployment permissions are protected.

Q. A Jenkins pipeline suddenly fails after a plugin or agent update. What do you do?

Suggested answer: First I identify the failing stage and compare the current run with the last successful run. I check Jenkins controller and agent logs, plugin compatibility, Java/runtime versions, agent connectivity, workspace permissions and tool paths. If the failure is caused by a recent change, I roll back the plugin/agent image or configuration through the approved change process. I rerun a small validation job before restoring the full pipeline. I preserve the failed build logs and document the root cause rather than repeatedly clicking Build.

Q. The Maven build passes but SonarQube quality gate fails. Do you deploy?

Suggested answer: No. I treat the quality gate as a release control. I inspect the SonarQube report for new bugs, vulnerabilities, code smells and coverage conditions. I share the finding with developers, track the fix, and rerun the pipeline against the corrected commit. A deliberate exception would need documented risk acceptance and approval under the team's policy; I would not silently bypass the gate.

Q. A deployment succeeds in Jenkins but users see an error. How do you investigate?

Suggested answer: I separate pipeline success from application success. I check the Deployment rollout, pod readiness and restarts, container logs, events, Service selectors/endpoints, Ingress routing, ConfigMaps/Secrets and dependency connectivity. I test the health endpoint from inside the cluster and from the expected external path. I compare the deployed image digest and configuration with the previous working release. If impact is high, I roll back to the last known-good image while investigating.

2. Kubernetes and Amazon EKS operations

Q. How would you upgrade a production Kubernetes/EKS cluster safely?

Suggested answer: I would treat it as a planned change, not a single version command. First I confirm the current control-plane and node versions, supported upgrade path, deprecated API usage, add-on compatibility, cluster capacity and application disruption budgets. I review release notes and back up critical configuration/state where appropriate. In EKS, I upgrade the control plane one supported minor version at a time, then update compatible add-ons such as VPC CNI, CoreDNS and kube-proxy. I create or update a node group with the target AMI/Kubernetes version, drain old nodes gradually, and allow workloads to reschedule. I check PodDisruptionBudgets, replicas, topology spread, storage and capacity before draining. Finally I validate nodes, system pods, ingress, DNS, networking, autoscaling and application health. I keep a rollback/recovery plan; EKS control-plane version downgrades are generally not the rollback mechanism, so recovery usually means restoring workloads/configuration or replacing node groups.

Q. What exact checks do you do before draining a node?

Suggested answer: I check which pods run on it, whether they are managed by Deployments/StatefulSets or are standalone, replica count, PodDisruptionBudget status, node capacity elsewhere, local storage usage, daemonsets and any special scheduling constraints. I confirm the remaining nodes can fit the workload. Then I cordon and drain the node using the approved process, monitor rescheduling, and only terminate/replace it after workloads are healthy.

Q. A pod is in CrashLoopBackOff. What is your troubleshooting sequence?

Suggested answer: I start with `kubectl get pods -n <namespace> -o wide`, then `kubectl describe pod <pod> -n <namespace>` to inspect events and probes. I check `kubectl logs <pod> -n <namespace>` and `kubectl logs <pod> -n <namespace> --previous` for the prior container attempt. I verify command/arguments, environment variables, ConfigMaps, Secrets, mounted volumes, image version, resource limits and dependency access. I fix the root cause and watch the rollout. I avoid deleting pods repeatedly without understanding why they restart.

Q. Pods are Running but the application is unreachable. What do you check?

Suggested answer: I verify readiness, Service selectors and endpoints first. Then I check target container port versus Service port/targetPort, NetworkPolicies, DNS, Ingress rules and controller health, load balancer target health, security groups and routing where applicable. I test connectivity hop by hop—from a pod, to the Service DNS name, then through Ingress or the load balancer. This narrows down whether the issue is the application, cluster networking or external AWS networking.

Q. What is the difference between Deployment, ReplicaSet and Service?

Suggested answer: A Deployment manages the desired application rollout and creates/updates ReplicaSets. A ReplicaSet maintains the required number of pod replicas. A Service provides a stable virtual IP/DNS name and selects matching pods. Common Service types are ClusterIP, NodePort and LoadBalancer. A Service does not create pods; it provides stable network access to them.

Q. How do you perform a zero/minimal-downtime application release on Kubernetes?

Suggested answer: I use multiple replicas, readiness and liveness probes, a rolling update strategy with suitable maxSurge/maxUnavailable values, and a PodDisruptionBudget where appropriate. I confirm capacity exists for surge pods. I deploy a new immutable image, watch `kubectl rollout status`, validate service endpoints and application health, and monitor errors/latency. If validation fails, I use `kubectl rollout undo deployment/<name> -n <namespace>` or redeploy the known-good image, then investigate.

Q. A node becomes NotReady in EKS. What do you do?

Suggested answer: I check `kubectl describe node`, node conditions, events and affected pods. In AWS I inspect the EC2 instance status checks, Auto Scaling Group or managed node group health, subnet IP capacity, security groups, IAM role, bootstrap/user-data and network path to the EKS API endpoint. I also check kubelet/container runtime and VPC CNI health. I protect availability by ensuring workloads have healthy replicas on other nodes, then replace or repair the node according to the node-group design.

3. Ansible automation, including Windows servers

Q. How would you structure Ansible for multiple environments?

Suggested answer: I would keep reusable roles for common configuration and separate inventory groups for Dev, Staging and Production. Variables that differ by environment go in `group_vars` or controlled inventory variables; secrets use Ansible Vault or an approved secrets manager. I use templates for configuration files, handlers to restart services only when configuration changes, and tags for controlled execution. I test changes against a non-production inventory first, use check mode/diff where supported, and promote the same reviewed playbook to higher environments.

Q. How would you deploy an application to Windows servers using Ansible?

Suggested answer: I would confirm WinRM connectivity and authentication first, then define Windows hosts in inventory with the correct connection settings. I would use Windows-specific modules such as `ansible.windows.win_ping`, `win_service`, `win_copy`, `win_template`, `win_package` and `win_updates` as needed. The playbook would validate prerequisites, copy or install the release, render environment-specific configuration, stop/start or restart the service safely, and verify service state and application health. I would use a limited batch/serial rollout to reduce impact, store credentials securely, and test in a non-production group first. I would not use Linux `apt` or `systemd` modules on Windows.

Q. What is Ansible idempotency and why does it matter?

Suggested answer: Idempotency means running the same playbook repeatedly should leave the system in the desired state without making unnecessary changes. For example, the `win_service` or `service` module ensures a service is started rather than blindly executing a start command every time. It makes deployments predictable and reduces configuration drift.

Q. A playbook works on one server but fails on the rest. How do you debug it?

Suggested answer: I use `--limit` to isolate a host, `-vvv` for detailed output, and inspect inventory variables, connectivity, permissions, OS differences, disk space, package versions and service state. I check whether the task is idempotent and whether a handler or template is causing the issue. I avoid exposing secrets in debug output. Once fixed, I test a small batch and then continue with a controlled rollout.

4. AWS infrastructure, networking and security

Q. An application in a private subnet cannot reach the internet or download packages. How do you troubleshoot?

Suggested answer: I verify the subnet's route table first. A private subnet normally routes internet-bound IPv4 traffic to a NAT Gateway in a public subnet; that public subnet needs a route to an Internet Gateway. I verify the NAT Gateway is available and has an Elastic IP, and check NACLs, security groups, DNS and the instance route table. For access to AWS services, I consider VPC endpoints so traffic can stay on the AWS network. I also check whether the application needs inbound internet access—private instances normally should not receive direct unsolicited inbound connections.

Q. An EC2 instance cannot access an S3 bucket. What do you check?

Suggested answer: I check whether the instance has the intended IAM role attached, whether the role policy permits the required S3 actions on the correct bucket/object ARN, and whether the bucket policy or KMS key policy denies access. I verify region, endpoint policy, VPC endpoint route/DNS and network connectivity. I use the instance role rather than storing long-lived access keys on the server. I test the exact operation and inspect CloudTrail or application error details.

Q. How do you secure an EKS workload's access to AWS services?

Suggested answer: I avoid putting static AWS keys in Kubernetes Secrets. I use IAM Roles for Service Accounts (IRSA) or EKS Pod Identity where supported, associate a narrowly scoped IAM role with the workload identity, and grant only the actions and resources it needs. I verify the trust relationship and service-account mapping, then test access from the pod. I also restrict Kubernetes RBAC and protect cluster secrets.

Q. How would you design a highly available AWS application environment?

Suggested answer: I would spread workloads across multiple Availability Zones, use an Application Load Balancer, run multiple application replicas on EKS/ECS or across EC2 Auto Scaling groups, and use Multi-AZ RDS where the database engine/design supports it. I keep application instances stateless where possible, store durable data in managed storage, configure health checks and autoscaling, and monitor with CloudWatch. I also define backup, restore and disaster-recovery objectives rather than assuming multi-AZ alone is a complete DR plan.

Q. How do you reduce AWS cost without reducing reliability?

Suggested answer: I start with cost and utilization data rather than guessing. I review EC2/EKS node CPU and memory, right-size instances, use autoscaling, remove unattached/unused resources, set S3 lifecycle policies, review NAT/data-transfer costs, and choose appropriate purchase options for steady workloads. For EKS I check bin-packing, requests/limits and node utilization. I validate changes with performance and availability metrics and keep a rollback option.

5. Terraform and infrastructure as code

Q. How do you manage Terraform for Dev, Staging and Production?

Suggested answer: I use reusable modules for common infrastructure and separate environment configurations with distinct variable files and state. Each environment should have isolated state and controlled credentials. I use a remote backend with state locking and encryption, protect state access, and run `terraform fmt`, `validate` and `plan` in CI before an approved `apply`. I review the plan carefully, especially for replacements or deletions, and apply only the reviewed plan. I do not allow multiple people or pipelines to modify the same state concurrently.

Q. Terraform plan shows that it will replace a production database. What do you do?

Suggested answer: I stop before apply. I inspect the plan and resource attributes marked ForceNew, compare configuration with the real resource, check provider/module changes and state drift, and confirm whether the change was intended. I protect critical resources with lifecycle rules such as `prevent_destroy` where appropriate, but I do not rely on that as a substitute for review. I discuss the impact, backup/restore plan and maintenance window, then correct the configuration or use an approved migration path.

Q. What happens if Terraform state is lost or becomes inconsistent?

Suggested answer: I stop further applies to avoid making the situation worse. I check the remote backend versioning and backups, identify the last known-good state, and compare it with actual infrastructure. If necessary, I restore state carefully or import existing resources using their real IDs. I run a refresh/plan and review all proposed changes before applying. State is sensitive and should be encrypted, access-controlled and versioned.

Q. How do you handle Terraform drift?

Suggested answer: I run a plan to identify differences between configuration/state and real infrastructure. I determine whether the change was an approved out-of-band update or accidental drift. If the change is desired, I update code to match; if not, I plan a controlled correction. I avoid blindly applying a plan until I understand destructive or replacement actions.

6. Monitoring, incidents and production support

Q. CPU usage is high and users report slowness. How do you investigate?

Suggested answer: I establish the impact and time window, then correlate application symptoms with CloudWatch metrics/logs and, where available, Prometheus/Grafana dashboards. I inspect CPU, memory, disk, network, pod restarts, throttling, request rate, latency and error rate. I identify whether the bottleneck is a pod, node, database or downstream dependency. I mitigate first—such as scaling within safe limits or rolling back a recent release—then investigate the root cause and document preventive actions.

Q. How do you handle a production incident?

Suggested answer: I follow the incident process: acknowledge and assess severity, communicate impact, assign an incident owner, and stabilize service. I use dashboards, logs, events and recent change history to narrow the cause. I prefer a safe mitigation such as rollback, failover or scaling over an untested production change. After recovery I verify user-facing health, communicate closure, and write an RCA with timeline, trigger, root cause, contributing factors, resolution and action owners.

Q. What would you monitor for EKS and application health?

Suggested answer: At cluster level I monitor node readiness, CPU/memory, pod capacity, API/server and system component health, scheduling failures and autoscaler activity. At workload level I monitor desired versus available replicas, restarts, pending pods, CPU/memory requests and limits, latency, traffic and error rates. At application level I monitor health endpoints and business-critical transactions. CloudWatch, Prometheus and Grafana can provide complementary infrastructure and workload views.

Q. A rollout is stuck with pods Pending. What do you check?

Suggested answer: I describe the pod and inspect events. Common causes include insufficient CPU/memory, node selectors or affinity that cannot be satisfied, taints/tolerations, PVC binding problems, quota limits and subnet IP exhaustion in EKS. I check node capacity and autoscaler events. I fix the scheduling constraint or capacity issue rather than simply lowering requests without understanding actual workload needs.

7. Azure-to-AWS migration and data-platform awareness

Q. The JD asks you to analyse Azure infrastructure and map it to AWS. How would you approach it?

Suggested answer: I would first inventory Azure subscriptions, resource groups, VNets/subnets, NSGs, VMs, managed identities, storage, databases, monitoring, DNS, load balancers and application dependencies. I would classify each workload by business criticality, dependencies, data sensitivity, availability, performance and migration constraints. Then I would create a target mapping—for example Azure VNet to AWS VPC, NSG to security groups/Network ACLs, VM to EC2, Blob Storage to S3, Azure Kubernetes Service to EKS, and Azure Monitor to CloudWatch—while checking differences rather than assuming one-to-one equivalence. I would define landing-zone security, connectivity, IAM, logging, backup, migration waves, testing, cutover and rollback. I would pilot a low-risk workload before production migration.

Q. How would you explain Azure networking compared with AWS VPC?

Suggested answer: Both use isolated virtual networks, subnets, route tables and security controls. AWS VPC maps broadly to Azure VNet; AWS subnet maps to an Azure subnet; AWS route tables map to Azure route tables; AWS security groups map broadly to Azure Network Security Groups. An AWS Internet Gateway is not a one-to-one object in Azure: Azure provides platform-managed internet connectivity and explicit outbound patterns such as NAT Gateway. AWS NAT Gateway and Azure NAT Gateway provide outbound internet translation for private subnet resources. VPN and dedicated private connectivity also have corresponding services, but routing, gateway placement and policy details differ.

Q. The JD mentions S3, Glue, Athena, Lake Formation, SQS/SNS and EventBridge. What should you know?

Suggested answer: S3 is object storage commonly used as a data lake foundation. AWS Glue provides data integration and catalog capabilities, including ETL jobs and the Data Catalog. Athena runs SQL queries directly against supported data in S3. Lake Formation helps centrally govern and grant fine-grained access to data lake data. SQS is a message queue for decoupling producers and consumers; SNS is pub/sub notification and fan-out; EventBridge routes events between producers and targets using rules. As a DevOps/platform engineer I would focus on IAM, networking, encryption, logging, deployment automation, retries, monitoring, quotas and cost controls around these services. My resume lists core AWS services but does not claim hands-on Glue/Athena/Lake Formation experience.

Q. How would you deploy a data ingestion service reliably?

Suggested answer: I would package the service as a versioned container or deploy it using the team's approved runtime, externalize configuration and secrets, and use CI checks before release. I would design retries with backoff, idempotent processing, dead-letter handling where supported, and clear queue/lag/error metrics. I would set resource requests/limits and autoscaling based on meaningful workload signals. I would ensure data is encrypted, access is least-privilege, and logs/metrics include correlation identifiers without exposing sensitive data.

8. Python, Linux and practical command questions

Q. How would you use Python in DevOps automation?

Suggested answer: I use Python for repeatable operational tasks such as calling REST APIs, validating deployment inputs, checking service health, parsing logs or generating reports. I would keep scripts parameterized, handle exceptions and timeouts, log useful context, avoid hard-coded credentials, and return non-zero exit codes when a pipeline step fails. For cloud access I would use an approved identity/role and SDK credentials chain rather than embedding keys.

Q. Which commands help you investigate Linux resource issues?

Suggested answer: I use `top` or `htop` for process usage, `free -h` for memory, `df -h` and `du -sh` for disk, `iostat`/`vmstat` where installed for I/O and system pressure, `ss -tulpn` for listening sockets, `systemctl status <service>` for service state, and `journalctl -u <service> --since '30 minutes ago'` for logs. I correlate the output with application and monitoring data before making changes.

Q. A Linux service will not start after a deployment. What do you do?

Suggested answer: I check `systemctl status`, `journalctl`, the service configuration and recent file changes. I validate permissions, ownership, environment variables, port conflicts, disk space, dependencies and configuration syntax. I compare with the last known-good release and roll back if production is affected. I restart only after correcting the cause, then verify process state, listening port and application health.

9. Behavioural and ownership scenarios

Q. Tell me about a production issue you handled.

Suggested answer: Use a real incident from your work and explain it in this order: business impact, symptoms, how you narrowed the issue, commands/logs/metrics checked, immediate mitigation, permanent fix and prevention. For example, if you handled a Kubernetes deployment issue, explain the actual pod/event/log/configuration evidence and how you verified recovery. Do not invent outage duration, percentage improvement or ownership you cannot substantiate.

Q. How do you work with developers and data engineers during a release?

Suggested answer: I agree on the release scope, artifact/version, configuration changes, dependencies, validation steps and rollback criteria. I make sure required approvals and change records are in place. During deployment I share status and evidence—pipeline stage, rollout state, health checks and monitoring. If validation fails, I communicate the impact and next action clearly rather than assuming a green pipeline means the release is healthy.

Q. The title is Senior DevOps Engineer. How do you handle a responsibility you have not owned before?

Suggested answer: I clarify the expected outcome, risk and constraints, review the current architecture and operational documentation, and identify the right subject-matter experts. I create a small test or proof of concept, validate it in a lower environment, document the runbook and then propose a controlled production change. I am comfortable taking ownership, but I do not claim prior production experience with a tool or service I have not operated.

10. Quick command reference

Jenkins / build

```
mvn clean test package
# Check pipeline logs, stage result, agent connectivity and credentials
```

Kubernetes health

```
kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl get events -A --sort-by=.lastTimestamp
```

Pod troubleshooting

```
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous
```

Deployment rollout

```
kubectl rollout status deployment/<name> -n <ns>
kubectl rollout history deployment/<name> -n <ns>
kubectl rollout undo deployment/<name> -n <ns>
```

Node maintenance

```
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
# Review PDBs, local storage and workload capacity before draining
```

Service connectivity

```
kubectl get svc,endpoints -n <ns>
kubectl describe ingress <name> -n <ns>
kubectl get networkpolicy -A
```

Terraform

```
terraform fmt -check
terraform init
terraform validate
terraform plan -out=tfplan
terraform show tfplan
# Apply only the reviewed plan with approval
```

Ansible

```
ansible-inventory -i inventory --graph
ansible all -i inventory -m ping
ansible-playbook -i inventory site.yml --check --diff
ansible-playbook -i inventory site.yml --limit <host> -vvv
```

Linux

```
top
free -h
df -h
ss -tulpn
systemctl status <service>
journalctl -u <service> --since '30 minutes ago'
```

11. Final interview checklist

- Be able to draw the Jenkins flow: GitHub → Jenkins → Maven tests → SonarQube quality gate → Docker build → registry/ECR → Dev → Staging → approval → Production → health validation.
- Be able to explain how one immutable image is promoted across environments.
- Practise an EKS upgrade explanation, including version compatibility, add-ons, node groups, PDBs, drain, validation and recovery.
- Practise one pod failure, one service-connectivity issue, one Pending pod and one NotReady node scenario using actual commands.

- Prepare one genuine Ansible example. If Windows Ansible is new to you, explain WinRM and Windows modules as knowledge, not as past production ownership.
- Prepare one Terraform state/drift example and explain remote state, locking, plan review and protection from accidental destroy.
- Prepare one AWS networking issue and one IAM/S3/EKS access issue.
- Be ready to discuss Azure-to-AWS mapping and data services at an architecture level; be transparent about services not used hands-on.
- Prepare two real incidents/releases from your own work with evidence, actions, outcome and lesson learned.
- Do not claim a specific scale, uptime, cost saving, team size or production ownership unless you can support it with a real example.

Source alignment: Prepared from the uploaded Sachin DevOps resume (2 pages) and Senior DevOps Engineer job description for Go Digital Technology Consulting LLP (3 pages). The technical explanations are interview preparation guidance; they should not be represented as additional work history unless personally performed.